

Visual Studio Code

Code (<https://code.visualstudio.com/>) is a cross-platform **text editor** developed by Microsoft, built on the **Electron** framework. *Visual Studio Code* is a binary distribution of the MIT-licensed **Code - OSS** (<https://github.com/microsoft/vscode/>) repository, with Microsoft specific customizations and released under a **proprietary license** (<https://code.visualstudio.com/license>). For details on the mixed licensing, see [this GitHub comment \(https://github.com/microsoft/vscode/issues/60#issuecomment-161792005\)](https://github.com/microsoft/vscode/issues/60#issuecomment-161792005). There is also a community-driven, MIT-licensed binary release called **VSCodium** (<https://vscodium.com/>) with **telemetry** (<https://code.visualstudio.com/docs/getstarted/telemetry>) disabled by default.

1 Installation

The following flavors of Visual Studio Code are available:

- **Code - OSS** — Official Arch Linux open-source release. Ships a configuration enabling **Open VSX** (<https://open-vsx.org/>).

<https://github.com/microsoft/vscode> || **code** (<https://archlinux.org/packages/?name=code>)

- **Visual Studio Code** — Proprietary Microsoft-branded release.

<https://code.visualstudio.com/> || **visual-studio-code-bin** (<https://aur.archlinux.org/packages/visual-studio-code-bin/>)^{AUR}

- **VSCodium** — Community open-source release. Nullifies telemetry in the source code [\[1\] \(https://github.com/VSCodium/vscodium/issues/267#issuecomment-542462446\)](https://github.com/VSCodium/vscodium/issues/267#issuecomment-542462446), also ships configuration with Open VSX.

<https://vscodium.com/> || **vscodium** (<https://aur.archlinux.org/packages/vscodium/>)^{AUR}

These different flavors are all built from the **Code - OSS repository** (<https://github.com/VSCodium/vscodium/blob/master/README.md>), but with different licensing and default configurations. Notably, only the proprietary builds are permitted to use Microsoft's marketplace and use Microsoft proprietary extensions such as the **OmniSharp C# Debugger** (<https://github.com/OmniSharp/omnisharp-vscode/blob/master/debugger.md>). The latter is enforced by a handshake mechanism, and cannot be circumvented. For more info on the differences between open source and proprietary "Visual Studio Code" branded builds, consult the **Code - OSS GitHub wiki** (<https://github.com/microsoft/vscode/wiki/Differences-between-the-repository-and-Visual-Studio-Code>).

1.1 Extensions support

One of Code's main strengths is its flexible API and rich extension ecosystem hosted on the [Visual Studio Marketplace \(https://marketplace.visualstudio.com/vscode\)](https://marketplace.visualstudio.com/vscode). However, the [terms of use of the marketplace \(https://aka.ms/vsmarketplace-ToU\)](https://aka.ms/vsmarketplace-ToU) only permit it to be used with the Microsoft branded releases. As a result, the Code - OSS source does not include a configured marketplace. The open-source releases above add the Open VSX extension registry, but this does not offer the same breadth of extensions. It is possible to bypass this limitation.

Warning

Some language extensions developed by Microsoft now strictly require the proprietary Visual Studio Code release to work [2] (<https://github.com/VSCodium/vscodium/issues/2300>). Tweaking `product.json` does not seem to help. Currently [C/C++ \(https://marketplace.visualstudio.com/items?itemName=ms-vscode.cpptools\)](https://marketplace.visualstudio.com/items?itemName=ms-vscode.cpptools) and [C# Dev Kit \(https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csdevkit\)](https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csdevkit) are known to be affected.

Known workarounds are:

- ask the maintainer to upload its extension to the [Open VSX registry \(https://open-vsx.org/\)](https://open-vsx.org/);
- add the Microsoft Visual Studio Code Marketplace by using [code-marketplace \(https://aur.archlinux.org/packages/code-marketplace/\)](https://aur.archlinux.org/packages/code-marketplace/)^{AUR} (or [vscodium-marketplace \(https://aur.archlinux.org/packages/vscodium-marketplace/\)](https://aur.archlinux.org/packages/vscodium-marketplace/)^{AUR} for VSCodium). The package installs a [Pacman hook](#) that patches `product.json` as shown in [this Github comment \(https://github.com/VSCodium/vscodium/issues/418#issuecomment-643664182\)](https://github.com/VSCodium/vscodium/issues/418#issuecomment-643664182) after every package update.

Tip

It is useful to enable a [keyboard shortcut to reload the IDE \(https://stackoverflow.com/a/69985500\)](https://stackoverflow.com/a/69985500) when experimenting with changing `product.json`.

2 Usage

Run `code` to start the application (or `codium` for VSCodium).

If for any reason you wish to launch multiple instances of Visual Studio Code, the `-n` flag can be used.

3 Configuration

3.1 Launch configuration

Per-user configuration can be set in the following files. Note that this is specific to these packages, as they use a patched loader script that reads these options.

Using these files, one can add flags for **Visual Studio Code** (https://code.visualstudio.com/docs/configure/command-line#_core-cli-options), **Electron** (<https://www.electronjs.org/docs/latest/api/command-line-switches>), or **Chromium** (<https://peter.sh/experiments/chromium-command-line-switches/>).

Package	Location
code (https://archlinux.org/packages/?name=code)	<code>~/.config/code-flags.conf</code>
visual-studio-code-bin (https://aur.archlinux.org/packages/visual-studio-code-bin/) ^{AUR}	<code>~/.config/code-flags.conf</code>
vscodium (https://aur.archlinux.org/packages/vscodium/) ^{AUR}	Not supported

3.2 Application configuration

Note

Packages like [code-features](https://aur.archlinux.org/packages/code-features/) (<https://aur.archlinux.org/packages/code-features/>)^{AUR} that patch `product.json` can override the `nameShort` and `dataFolderName` variables, making the below section inaccurate.

VSC stores settings in `$XDG_CONFIG_HOME/nameShort/User/settings.json` and the extension data in an **XDG-nonstandard** (<https://github.com/microsoft/vscode/issues/3884>) `$HOME/dataFolderName` directory, using variables from `product.json`.

This translates to the following default paths in various versions:

- [code](https://archlinux.org/packages/?name=code) (<https://archlinux.org/packages/?name=code>) stores settings in `~/.config/Code - OSS/User/settings.json` and extensions in `~/.vscode-oss`
- [visual-studio-code-bin](https://aur.archlinux.org/packages/visual-studio-code-bin/) (<https://aur.archlinux.org/packages/visual-studio-code-bin/>)^{AUR} stores settings in `~/.config/Code/User/settings.json` and extensions in `~/.vscode`
- [vscodium](https://aur.archlinux.org/packages/vscodium/) (<https://aur.archlinux.org/packages/vscodium/>)^{AUR} stores settings in `~/.config/VSCodium/User/settings.json` and extensions in `~/.vscode-oss`

When migrating from Code to Codium (or vice versa), the settings directory can be copied or moved. Since they share most of their codebase, the settings are compatible.

3.3 Integrated terminal

View > *Integrated Terminal* or Ctrl + ` opens up an integrated terminal. By default, [Bash](#) is used with no additional arguments, although this can be changed. `terminal.integrated.shell.linux` sets the default shell to be used and `terminal.integrated.shellArgs.linux` sets the arguments to be passed to the shell.

Example:

```
~/ .config/Code/User/settings.json
```

```
"terminal.integrated.shell.linux": "/usr/bin/fish",  
"terminal.integrated.shellArgs.linux": ["-l", "-d 3"]
```

You might face weird prompts after setting the integrated shell arguments with an external terminal. Remove the line to solve the problem or use an external terminal.

3.4 External terminal

If you are using [Terminator](#) as default terminal for Arch and you have an error on Visual Studio Code:

```
Unable to launch debugger worker process (vsdbg) through the terminal.  
spawn truecolor ENOENT
```

, you can change the terminal that will be used by Visual Studio to another terminal (e.g. [gnome-terminal](https://archlinux.org/packages/?name=gnome-terminal) (<https://archlinux.org/packages/?name=gnome-terminal>)).

```
"terminal.external.linuxExec": "Your alternative terminal"
```

 sets the default terminal to be used for exec debug.

Example:

```
~/ .config/Code/User/settings.json
```

```
"terminal.external.linuxExec": "gnome-terminal"
```

3.5 Running natively under Wayland

Visual Studio Code uses Electron, see [Wayland#Electron](#) for more information on how to run it natively under Wayland.

Keep in mind that some applications bring their own copy of Electron and will not read the standard Electron flags file. You can set flags separately in each application's [configuration file](#).

[vscodium](https://aur.archlinux.org/packages/vscodium/) (<https://aur.archlinux.org/packages/vscodium/>)^{AUR} does not load config files. It provides a dedicated `vscodium-wayland.desktop` [desktop entry](#) file, which appears as "VSCodium - Wayland" in the menu.

3.6 Native file dialog

If using Plasma, by default VS Codium opens GTK file dialogs. To fix that, ensure that KDE [xdg-desktop-portal](https://archlinux.org/packages/?name=xdg-desktop-portal-kde) ([xdg-desktop-portal-kde](https://archlinux.org/packages/?name=xdg-desktop-portal-kde)) is installed and set the `GTK_USE_PORTAL=1` [environment variable](#).

3.7 Remote SSH

The official "Remote - SSH" extension is not working with [code](https://archlinux.org/packages/?name=code) (VSCode-OSS).

There are two options:

- install [code-features](https://aur.archlinux.org/packages/code-features/) ^{AUR}
- install the [Open Remote-SSH extension](https://github.com/jeanp413/open-remote-ssh) (there is a [PR to make it work with VSCode-OSS](https://github.com/jeanp413/open-remote-ssh/pull/189)).

4 Troubleshooting

4.1 Global menu not working in KDE/Plasma

Visual Studio Code uses DBus to pass the menu to Plasma, try installing [libdbusmenu-glib](https://archlinux.org/packages/?name=libdbusmenu-glib) (<https://archlinux.org/packages/?name=libdbusmenu-glib>). [\[3\]](https://github.com/microsoft/vscode/issues/34510#issuecomment-364896975) (<https://github.com/microsoft/vscode/issues/34510#issuecomment-364896975>)

4.2 Unable to move items to trash

By default, [Electron](https://www.electronjs.org/) applications use `gio` to delete files. `kioclient5` is automatically selected instead if [Plasma](#) is detected. Different trash implementations can be used by setting the `ELECTRON_TRASH` [environment variable](#).

For example, for deleting files using [trash-cli](https://archlinux.org/packages/?name=trash-cli):

```
$ ELECTRON_TRASH=trash-cli code
```

At the time of writing, Electron supports `kioclient5`, `kioclient`, `trash-cli`, `gio` (default) and `gvfs-trash` (deprecated). More info is available at this [documentation page](https://github.com/electron/electron/blob/master/docs/api/environment-variables.md#electron-trash-linux) (<https://github.com/electron/electron/blob/master/docs/api/environment-variables.md#electron-trash-linux>).

4.3 Unable to debug C#

If you want to debug C#.NET (using the [OmniSharp extension \(https://www.omnisharp.net\)](https://www.omnisharp.net)) then you need to install the Microsoft branded release (from the AUR). This is apparently because the .NET Core debugger is only licensed to be used with official Microsoft products - see [this github discussion \(https://github.com/OmniSharp/omnisharp-vscode/issues/1431#issuecomment-297578930\)](https://github.com/OmniSharp/omnisharp-vscode/issues/1431#issuecomment-297578930).

When using the open-source package, debugging fails fairly quietly. The debug console will just show the initial message:

```
You may only use the Microsoft .NET Core Debugger (vsdbg) with
Visual Studio Code, Visual Studio or Visual Studio for Mac software
to help you develop and test your applications.
```

For debugging with the open-source package, [netcoredbg \(https://aur.archlinux.org/packages/netcoredbg/\)](https://aur.archlinux.org/packages/netcoredbg/)^{AUR} can be used. To run it in VS Code, add this configuration to .NET Core launch configuration of the project:

```
./.vscode/launch.json

"configurations": [
  {
    ...
    "pipeTransport": {
      "pipeCwd": "${workspaceFolder}",
      "pipeProgram": "/usr/bin/bash",
      "pipeArgs": ["-c"],
      "debuggerPath": "/usr/bin/netcoredbg"
    }
    ...
  ]
}
```

This is stable, actively maintained, and the preferred method for open-source VS Code users.

4.4 Unable to open .csproj with OmniSharp server, invalid Microsoft.Common.props location

You have to switch from mono to proper SDK version props.

```
/opt/dotnet/sdk/{VERSION}/Sdks/Microsoft.NET.Sdk/Sdk/Sdk.props

$(MSBuildExtensionsPath)\$(MSBuildToolsVersion)\Microsoft.Common.props
```

Modify import to look like this:

```
/opt/dotnet/sdk/{VERSION}/Sdks/Microsoft.NET.Sdk/Sdk/Sdk.props

/opt/dotnet/sdk/{VERSION}/Current/Microsoft.Common.props
```

4.5 Error from OmniSharp that MSBuild cannot be located

It is noted in the [OmniSharp introduction \(https://github.com/OmniSharp/omnisharp-roslyn#introduction\)](https://github.com/OmniSharp/omnisharp-roslyn#introduction) that Arch Linux users should install the [mono-msbuild \(https://archlinux.org/packages/?name=mono-msbuild\)](https://archlinux.org/packages/?name=mono-msbuild) package. Without it, you might get an error like:

OmniSharp Log

```
[info]: OmniSharp.MSBuild.Discovery.MSBuildLocator
Registered MSBuild instance: StandAlone 15.0 - "~/vscode/extensions/ms-vscode.csharp-1.18.0/.omnisharp/1.32.11/omnisharp/msbuild/15.0/Bin"
MSBuildExtensionsPath = /usr/lib/mono/xbuild
BypassFrameworkInstallChecks = true
CscToolPath = ~/.vscode/extensions/ms-vscode.csharp-1.18.0/.omnisharp/1.32.11/omnisharp/msbuild/15.0/Bin/Roslyn
CscToolExe = csc.exe
MSBuildToolsPath = ~/.vscode/extensions/ms-vscode.csharp-1.18.0/.omnisharp/1.32.11/omnisharp/msbuild/15.0/Bin
TargetFrameworkRootPath = /usr/lib/mono/xbuild-frameworks
System.TypeLoadException: Could not load type of field 'OmniSharp.MSBuild.ProjectManager:_queue' (13) due to: Could not load file or assembly 'System.Threading.Tasks.Dataflow, Version=4.5.24.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a' or one of its dependencies.
...
```

You might be able to build anyway (possibly depending whether you have [mono \(https://archlinux.org/packages/?name=mono\)](https://archlinux.org/packages/?name=mono) installed too).

Omnisharp ships with its own mono version, so, if it is unable to locate the installed one, if you want to tell omnisharp to look for a "global" mono installed in your machine, put this in your settings.json:

settings.json

```
"omnisharp.useGlobalMono": "always"
```

4.6 Saving with "Retry as Sudo" does not work

This feature does not work in the [code \(https://archlinux.org/packages/?name=code\)](https://archlinux.org/packages/?name=code) package, because Microsoft does not support the way the Arch package is packaged (native instead of bundled Electron). See [FS#61516 \(https://bugs.archlinux.org/task/61516\)](https://bugs.archlinux.org/task/61516) and the [upstream bug report \(https://github.com/Microsoft/vscode/issues/70403\)](https://github.com/Microsoft/vscode/issues/70403) for more information.

The binary release [visual-studio-code-bin \(https://aur.archlinux.org/packages/visual-studio-code-bin/\)](https://aur.archlinux.org/packages/visual-studio-code-bin/)^{AUR} does not have this issue, and the feature works there.

4.7 Keyboard variants or keymappings do not map

As per the [wiki on GitHub \(https://github.com/Microsoft/vscode/wiki/Keybinding-Issues#troubleshoot-linux-keybindings\)](https://github.com/Microsoft/vscode/wiki/Keybinding-Issues#troubleshoot-linux-keybindings):

Switching keyboard layouts under some Linux window managers does not result in a change in the low level X window APIs VS Code uses to read the current keyboard layout. This means that

VS Code ends up sometimes reading one of the other configured keyboard layouts and not the current active one. PR welcome...

Per the wiki, there are two possible solutions:

1. make sure `setxkbmap -query` returns as the first keyboard layout the one you want to work with in VS Code.
2. use `"keyboard.dispatch": "keyCode"` in your settings and restart VS Code. This will prevent VS Code from trying to determine your keyboard layout whatsoever.

4.8 Command ".." not found

In the Microsoft branded releases, the `product.json` file lists the extensions that are allowed to use certain proposed APIs accessed by extensions. Code - OSS and VSCodium distributions lack these values, though this does not appear to be due to licensing. Unlike the forced Marketplace enabling, this workaround is endorsed by Microsoft [\[4\] \(https://docs.microsoft.com/en-us/visualstudio/liveshare/reference/linux#vs-code-oss-issues\)](https://docs.microsoft.com/en-us/visualstudio/liveshare/reference/linux#vs-code-oss-issues).

This issue can be resolved by installing a Pacman hook that patches the file on every package update:

- For [code](https://archlinux.org/packages/?name=code) (<https://archlinux.org/packages/?name=code>), install [code-features](https://aur.archlinux.org/packages/code-features/) (<https://aur.archlinux.org/packages/code-features/>)^{AUR}
- For [vscodium](https://aur.archlinux.org/packages/vscodium/) (<https://aur.archlinux.org/packages/vscodium/>)^{AUR}, install [vscodium-features](https://aur.archlinux.org/packages/vscodium-features/) (<https://aur.archlinux.org/packages/vscodium-features/>)^{AUR}

You can also manually add the relevant entries to the `extensionAllowedProposedApi` section in the `product.json` file:

- For [code](https://archlinux.org/packages/?name=code) (<https://archlinux.org/packages/?name=code>), edit `/usr/lib/code/product.json`
- For [vscodium](https://aur.archlinux.org/packages/vscodium/) (<https://aur.archlinux.org/packages/vscodium/>)^{AUR}, edit `/usr/share/vscodium/resources/app/product.json`

An example of a manual configuration that would make Live Share work is [\[5\] \(https://docs.microsoft.com/en-us/visualstudio/liveshare/reference/linux#vs-code-oss-issues\)](https://docs.microsoft.com/en-us/visualstudio/liveshare/reference/linux#vs-code-oss-issues):

`product.json`

```
...
"extensionAllowedProposedApi": [
  "ms-vs-liveshare.vsliveshare",
  "ms-vscode.node-debug",
  "ms-vscode.node-debug2"
]
...
```

Finally, you can also enable these options using command line flags, [as described for the GitHub pull request extension \(https://github.com/Microsoft/vscode-pull-request-github/wiki#3-why-isnt-the-extension-starting\)](https://github.com/Microsoft/vscode-pull-request-github/wiki#3-why-isnt-the-extension-starting).

4.8.1 VS Live Share missing API

Use either the solution above by editing the `product.json`, or open VS Code with:

```
$ code --enable-proposed-api ms-vsliveshare.vsliveshare
```

Also note that for this extension to work, you need to install dependencies listed here [\[6\] \(https://docs.microsoft.com/en-us/visualstudio/liveshare/reference/linux#tips-for-community-supported-distros\)](https://docs.microsoft.com/en-us/visualstudio/liveshare/reference/linux#tips-for-community-supported-distros).

4.8.2 Command 'remote-containers.openFolder' not found

Open VS Code enabling remote-containers API as commented in [FS#63374 \(https://bugs.archlinux.org/task/63374\)](https://bugs.archlinux.org/task/63374):

```
$ code-oss --enable-proposed-api ms-vscode-remote.remote-containers
```

4.8.3 Command 'GitHub Pull Requests: Configure Remotes...' resulted in an error (command 'pr.configureRemotes' not found)

Open VS Code with:

```
$ code --enable-proposed-api GitHub.vscode-pull-request-github
```

4.9 Git: ssh_askpass: exec(/usr/lib/ssh/ssh-askpass): No such file or directory

This error is a result of an encrypted ssh-key and inability to use ssh agent, see [bug report \(https://github.com/microsoft/vscode/issues/57488\)](https://github.com/microsoft/vscode/issues/57488). The issue can be solved by installing a dialogue provider like [SSH keys#x11-ssh-askpass](https://archlinux.org/packages/?name=ksshaskpass) or the alternatives listed there (e.g. [ksshaskpass \(https://archlinux.org/packages/?name=ksshaskpass\)](https://archlinux.org/packages/?name=ksshaskpass) for KDE).

One thing to note is that for `ksshaskpass` you would either need to link it from `/usr/lib/ssh/ssh-askpass` to get VSCode to find it:

```
# ln /usr/bin/ksshaskpass /usr/lib/ssh/ssh-askpass
```

or set the following [environment variables](#) for your shell, see [\[7\] \(https://github.com/microsoft/vscode/issues/57488#issuecomment-1413903793\)](https://github.com/microsoft/vscode/issues/57488#issuecomment-1413903793):

```
GIT_ASKPASS=ksshaskpass
SSH_ASKPASS=ksshaskpass
SSH_ASKPASS_REQUIRE=prefer
```

To disable VSCode's internal `git-askpass`, add:

```
~/config/Code - OSS/User/settings.json
```

```
{  
  "git.useIntegratedAskPass": false  
}
```

4.10 Cutoff characters in integrated Terminal

Characters that are too wide can end up clipping. For example the italic bold text of Deno stack-traces.

This can be avoided by setting "terminal.integrated.rendererType" to "experimentalWebgl".

4.11 Blurry text under Wayland

Visual Studio Code defaults to run under Xwayland, which may cause blurry text if you are using HiDPI screens. To fix this issue, try forcing Electron to run under Wayland—see [#Running natively under Wayland](#).

Alternatively, if your Wayland environment provides the option to run Xwayland applications unscaled, you can circumvent this issue. You may then run Visual Studio Code with the `--force-device-scale-factor=` option to achieve an appropriate scale for your screen.

For example, for a scaling factor of 2:

```
$ code --force-device-scale-factor=2
```

4.12 No such interface“org.freedesktop.Secret.Collection”

See [settings-sync#_troubleshooting-keychain-issues \(https://code.visualstudio.com/docs/editor/settings-sync#_troubleshooting-keychain-issues\)](#)

4.13 Authentication with Github failed while using VSCodium

When connecting a Github account, change "vscodium" to "vscode" in the URL as seen in [this comment \(https://github.com/VSCodium/vscodium/issues/401#issuecomment-631502370\)](#). Then copy the identification token into VSCodium. Should it still fail, install a keyring like [gnome-keyring \(https://archlinux.org/packages/?name=gnome-keyring\)](#) or create a new keyring as mentioned here [in the Visual Studio Code docs \(https://code.visualstudio.com/docs/editor/settings-sync#_troubleshooting-keychain-issues\)](#) and here [on Github \(https://github.com/microsoft/vscode/issues/92972#issuecomment-625751232\)](#).

4.14 An OS keyring couldn't be identified

On some desktop environments like i3, VSCode fails to detect the keyring. If you are using gnome-keyring, you can add the following line to force VSCode to use that keyring:

```
~/vscode-oss/argv.json
```

```
{
  ...
  "password-store": "gnome-libsecret",
}
```

The example path above is for the official [code](https://archlinux.org/packages/?name=code) (<https://archlinux.org/packages/?name=code>) package. You might have to adjust the `.vscode-oss` directory path depending if you have a different one installed.

4.15 Interface fonts differ from the fonts chosen in the system settings

To render its interface VSCode uses the [fonts](#) chosen in the system settings. If you are using OTF fonts, try TTF ones.

Retrieved from "https://wiki.archlinux.org/index.php?title=Visual_Studio_Code&oldid=864954"